códigofacilito

Clase 13 · AI Engineer Bootcamp

Model Context Protocol (MCP)

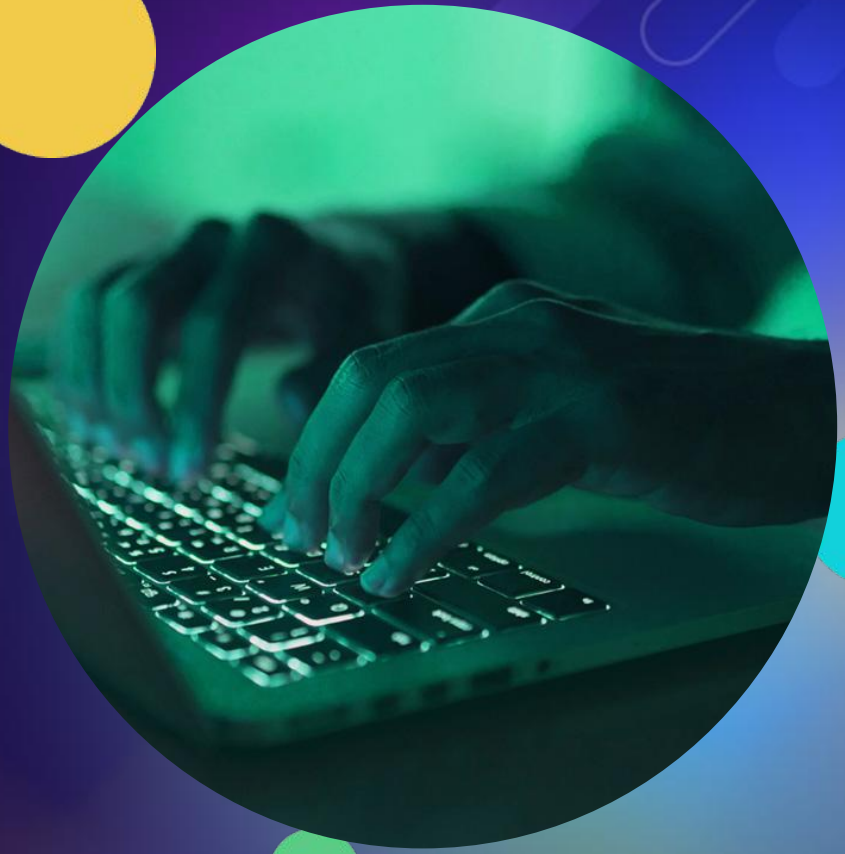
Ramsés Camas · Bootcamp AI Engineer

El artefacto de la clase 🎯

Al final de esta clase tendrás dos archivos funcionando: mcp/server.py, un servidor que expone herramientas reutilizables bajo el protocolo estándar, y mcp/client.py, un cliente que consume esas herramientas y las conecta con un LLM. La idea central: desacoplar las herramientas del agente.

códigofacilito

El cuello de botella M×N



Con M aplicaciones de IA y N fuentes de datos, sin un estándar terminas escribiendo M×N integraciones personalizadas. Cada cambio de framework rompe todo. Cada cambio de API externa también. Tus herramientas viven atrapadas dentro de tu aplicación y no son reutilizables.

códigofacilito

De M×N a M+N

MCP convierte el problema en M+N. Cada aplicación habla MCP. Cada herramienta habla MCP. Todos se entienden.

- **Origen** Anthropic, noviembre 2024. Protocolo abierto con especificación pública.
- **Open source** SDKs oficiales en Python, TypeScript, Java, Kotlin, C# y más.
- **Adopción por clientes** Claude Desktop, Cursor, Cline, Zed, Windsurf y más.
- **Adopción por frameworks** LangGraph, n8n, OpenAI Agents SDK integran MCP nativamente.
- **Ecosistema** Cientos de servidores comunitarios disponibles públicamente.

“MCP es el USB-C de los agentes de IA”

— Analogía oficial de Anthropic

códigofacilito

MCP ≠ Function Calling

Function Calling

Es una capacidad del LLM que le permite invocar herramientas externas. Es una feature del modelo: saber hablar por teléfono.

MCP

Es el protocolo que permite que las herramientas vivan fuera de tu aplicación y sean reutilizables entre aplicaciones. Es la red telefónica sobre la que corre function calling.

códigofacilito

Los 3 roles

La arquitectura de MCP se compone de tres roles bien definidos. Un Host puede tener múltiples Clients. Cada Client habla con un Server distinto.

- **Host** La aplicación principal con la que el usuario interactúa. Contiene al LLM. Ejemplos: Claude Desktop, Cursor IDE, tu agente custom.
- **Client** Vive dentro del Host. Mantiene una conexión 1:1 con un servidor MCP y traduce entre el formato del protocolo y lo que el LLM espera.
- **Server** Programa independiente y ligero que expone capacidades (herramientas, datos, prompts) bajo el estándar MCP.
- **Relación** Un Host, muchos Clients. Un Client, un Server. Arquitectura limpia y desacoplada.
- **Analogía** El Host es el usuario del teléfono, el Client es el cable, el Server es el otro lado de la línea.

Tools, Resources, Prompts

Las tres primitivas del servidor

/01 Tools

Funciones que el modelo puede invocar. Pueden tener efectos colaterales. Ejemplo: enviar email, escribir en DB, calcular métricas.

/02 Resources

Datos de solo lectura que el servidor expone. La aplicación o el usuario deciden usarlos. Ejemplo: archivos, esquemas, logs.

/03 Prompts

Plantillas reutilizables que el usuario puede invocar. Viven en el servidor. Ejemplo: instrucciones para analizar una tabla.

/04 Regla útil

Si el LLM decide cuándo usarlo → Tool. Si el usuario lo selecciona → Prompt. Si la app lo adjunta como contexto → Resource.

códigofacilito

Ciclo de vida

Todo el intercambio va en JSON-RPC 2.0. Si conoces cómo funciona un Language Server Protocol en un IDE, esto te va a sonar familiar. No es coincidencia: MCP se inspiró explícitamente en LSP.

- **1. Inicialización** Cliente y servidor se saludan y negocian capabilities: qué puede hacer cada uno.
- **2. Listado** El cliente pregunta al servidor qué tools, resources y prompts expone.
- **3. Intercambio** El cliente invoca tools o lee resources según el razonamiento del LLM.
- **4. Terminación** La sesión se cierra de forma controlada cuando termina la conversación.
- **Protocolo** JSON-RPC 2.0 sobre stdio o HTTP. El SDK maneja toda la serialización por ti.

¿Cómo viajan los mensajes?

stdio (local): el host arranca el servidor como subproceso y se comunican por stdin/stdout. Caso de uso: servidores personales, asistentes locales, desarrollo. Simple y directo. Sin red, sin puertos, sin Docker. Es lo que usa Claude Desktop por defecto y lo que vamos a usar en clase hoy.

Streamable HTTP (remoto): el servidor vive en otro lugar (contenedor, nube) y se comunica por HTTP con capacidad de streaming. Caso de uso: servidores compartidos por equipos, servicios SaaS, producción. En producción, para escalar a equipos, migras a HTTP sin cambiar tu lógica de tools.

Anatomía del servidor 🎯

Un servidor MCP en Python tiene tres cosas esenciales: una instancia del servidor (usamos FastMCP del SDK oficial), funciones decoradas como tools con type hints y docstrings descriptivos, y un punto de arranque que elige el transport. El SDK se encarga del handshake, el listado de capabilities, la serialización JSON-RPC y el manejo de errores. Tú solo te enfocas en la lógica de negocio.

códigofacilito
Los docstrings son tu API al modelo

códigofacilito

Tools vs Resources: ¿cuándo usar cuál?

Tool

El LLM decide cuándo invocarla. Útil cuando el agente debe razonar sobre qué necesita. Los agentes autónomos consumen principalmente este tipo de capacidad.

Resource

La aplicación o el usuario deciden cuándo adjuntarlo al contexto. Útil cuando quien decide es el humano, no el modelo. Los asistentes interactivos los consumen principalmente.

códigofacilito

MCP seguro

Un servidor MCP corre con los permisos del proceso que lo lanza. Si tu servidor tiene acceso a tu filesystem, el agente también. Principio de mínimos permisos siempre.

- **Prompt injection** Un archivo malicioso expuesto como resource puede contener instrucciones ocultas que secuestren al agente.
- **Credenciales** Nunca hardcodees tokens. Usa variables de entorno o el sistema de auth del host.
- **Tools destructivas** Operaciones irreversibles (borrar, mutar DB, enviar correos) requieren confirmación explícita.
- **Validación de inputs** Valida estrictamente todos los argumentos antes de ejecutar. El LLM puede equivocarse.
- **Mínimos permisos** Concede al servidor solo lo que necesita. Filesystem limitado, DB de solo lectura cuando aplique.

El Inspector MCP

🎯

Probar un servidor ejecutándolo manualmente es un infierno. Para eso existe el Inspector oficial: una interfaz web que se conecta a tu servidor y te permite listar las tools, resources y prompts disponibles, invocar tools con parámetros arbitrarios y ver la respuesta cruda del servidor sin gastar tokens del LLM. Lo lanzas con: npx @modelcontextprotocol/inspector.

Anatomía de un cliente MCP

Cuatro responsabilidades

/01 Conectar

Establecer la conexión al servidor (stdio o HTTP) y completar el handshake inicial de capabilities.

/02 Descubrir

Listar las tools, resources y prompts disponibles en el servidor al iniciar la sesión.

/03 Traducir

Convertir el formato MCP al formato de tool-use que espera la API del LLM (Anthropic, OpenAI, etc).

/04 Orquestar

Manejar el loop: cuando el LLM pide una tool, rutearla al servidor y devolver el resultado al modelo.

Claude Desktop

🎯

Claude Desktop es un cliente MCP completo. Para conectarle un servidor, editas claude_desktop_config.json y agregas la entrada. Dos cosas a cuidar: usar rutas absolutas al binario (uv, python, etc.) y especificar el directorio de trabajo explícitamente. El 90% de los errores de primeros servidores vienen de rutas relativas mal configuradas. Una vez guardado y reiniciado Claude Desktop, las tools aparecen dentro del chat.

códigofacilito

Loop cliente

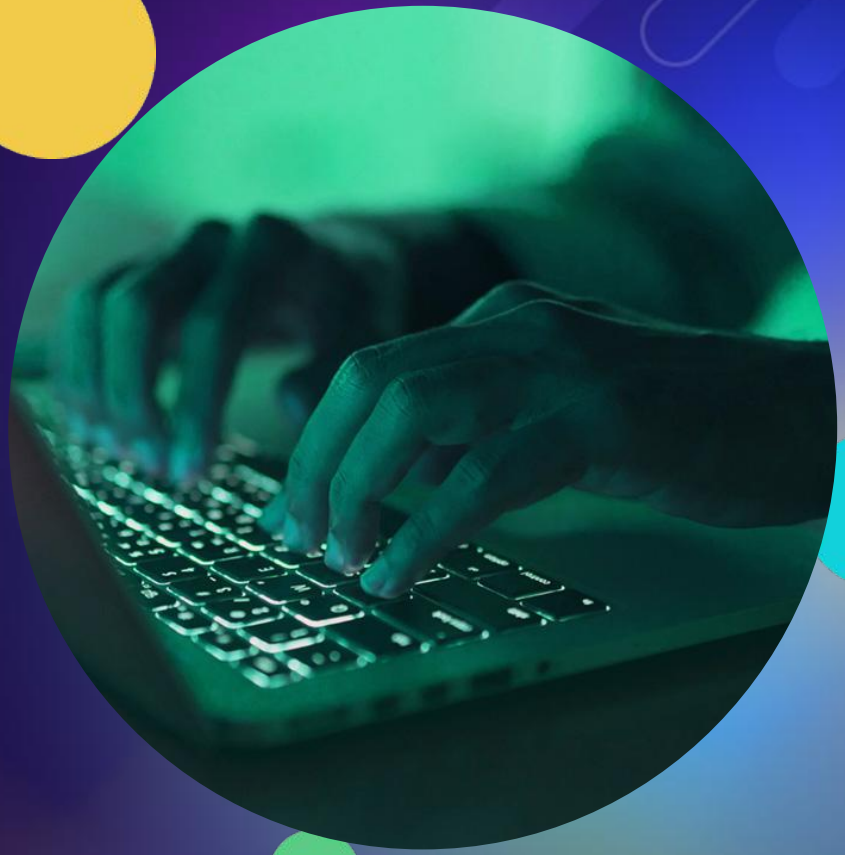
Cuando construyes tu propio agente (como el DocOps Agent del capstone), necesitas tu propio cliente. Este es el loop que ejecutarás en la Práctica 3.

- **1. Conectar** El cliente se conecta a los servidores MCP y obtiene la lista de tools disponibles.
- **2. Enviar query** El cliente pasa la pregunta del usuario + las tools a la API de LLM.
- **3. Razonar** LLM decide qué tool invocar y con qué parámetros. Devuelve un bloque tool_use.
- **4. Ejecutar** El cliente rutea la invocación al servidor correcto, espera el resultado y se lo devuelve a LLM.
- **5. Repetir** El loop continúa hasta que LLM genere una respuesta final sin pedir más tools.

códigofacilito

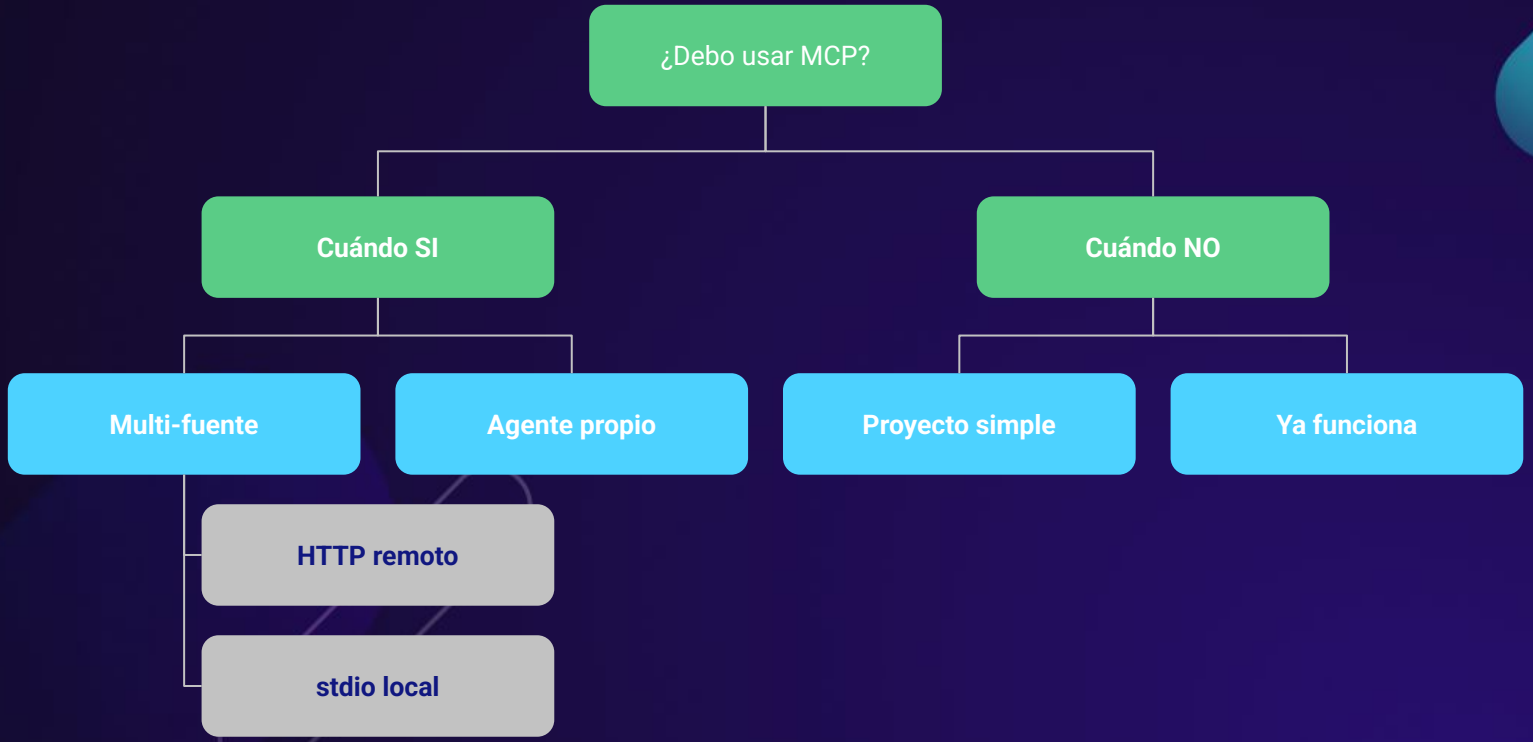
Un host, muchos servidores

Un host MCP bien diseñado mantiene conexiones simultáneas con múltiples servidores. El LLM ve todas las tools juntas y decide cuál usar. Ejemplo: un agente conectado a filesystem, SQLite y búsqueda semántica. Aquí es donde MCP brilla.



códigofacilito

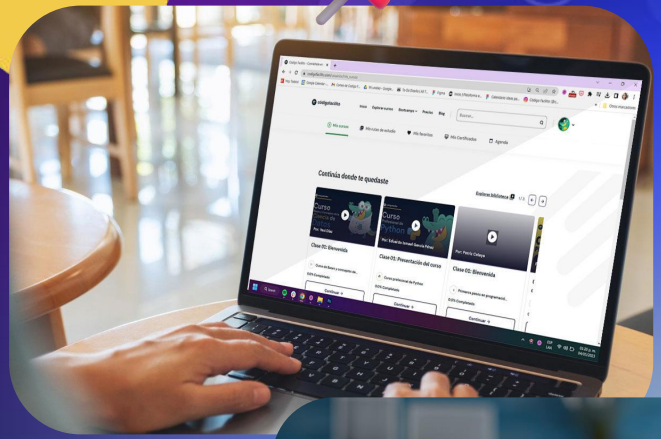
¿Debo usar MCP?



códigofacilito

/01 Prácticas

Tres ejercicios para construir MCP hoy



códigofacilito

Práctica 1: Hola Mundo 🎯

Objetivo: matar la fricción de setup antes de hacer algo complejo. Construirás un servidor con dos tools: una que devuelve la hora actual en una zona horaria, y otra que hace una operación matemática.

códigofacilito

Práctica 2

Construirás un servidor MCP que consulte una API pública (OpenWeather) o una base SQLite local con datos de ejemplo.

- **Diseño** ¿Qué tools expones? ¿Cómo las describes para que el modelo las use bien? ¿Qué errores devuelves?
- **Docstrings** Descripciones claras de qué hace cada tool, cuándo usarla y cuándo NO. Este es el 80% del éxito.
- **Validación** Valida inputs con type hints y maneja errores de red sin crashear el servidor.
- **Testing** Prueba cada tool en el Inspector antes de conectar al cliente final.
- **Éxito** Claude resuelve una pregunta real combinando al menos dos de tus tools.

códigofacilito

Práctica 3: Cliente

🎯

Escribirás un cliente MCP en Python desde cero (con el SDK oficial) que se conecta al servidor de la Práctica 2, lista tools, las transforma al formato de la API de Anthropic, inicia una conversación con Claude y maneja el loop de tool-use hasta terminar. Al final entenderás lo que Claude Desktop hace por dentro. Deja de ser magia. Éxito: tu client.py responde preguntas usando tus tools, desde la terminal, sin UI.

códigofacilito

Al capstone

El DocOps Agent que construirás como proyecto final puede arquitectarse como un servidor MCP. Lo que construyas hoy no es un ejercicio desechable: es la base arquitectónica del capstone.

- **Desde Claude Desktop** Usarás tu DocOps Agent directamente dentro del chat de Claude.
- **Desde Cursor** Lo invocarás mientras programas, sin salir de tu IDE.
- **Desde tu cliente custom** Lo integrarás en agentes propios con LangGraph u OpenAI SDK.
- **Compartir con el bootcamp** Otros estudiantes podrán conectar tu servidor a sus propios clientes.
- **Portabilidad total** Un servidor, muchos clientes. Una sola integración vale para todos.

El ecosistema es tuyo

Recursos para seguir aprendiendo

/01 Docs

modelcontextprotocol.io — documentación oficial completa del protocolo.

/02 Python SDK

github.com/modelcontextprotocol/python-sdk — el SDK que usamos hoy.

/03 Servers

github.com/modelcontextprotocol/servers — servidores oficiales: filesystem, SQLite, GitHub y más.

/04 Inspector

npx @modelcontextprotocol/inspector — tu mejor herramienta de debugging.